Review request: the new `git-vista-mcp` crate (M2.23a, #245)

You are reviewing a freshly merged crate in the public repo `tom2025b/git-vista`. Use your GitHub connector to read the real files — everything you need is on `main`:

- `crates/git-vista-mcp/src/main.rs` — stdio JSON-RPC 2.0 dispatch (MCP)
- `crates/git-vista-mcp/src/auth.rs` — bootstrap-token → session exchange
- `crates/git-vista-mcp/src/http.rs` — hand-rolled HTTP/1.1 over TcpStream
- `crates/git-vista-mcp/src/tools.rs` — tool surface + 401-retry logic
- `crates/git-vista-mcp/tests/live_handshake.rs` — the live integration test
- `crates/git-vista-mcp/Cargo.toml` — the deliberately tiny dependency surface

Context worth having open beside them:

- `crates/git-vista-server/src/security.rs` — the Host/Origin/CSRF gate the bridge's wire posture must satisfy
- `crates/git-vista-server/src/session.rs` — the single-use, self-replacing bootstrap token this crate consumes
- Issue #245 — the scope and acceptance criteria it was built against

What this crate is

The first slice of #153: an MCP stdio bridge so an agent can drive git-vista through the same HTTP API (and eventually the same reviewed-plan funnel) the browser uses — never through raw git argv. This slice is read-only by design: one tool, `list_repositories`. The write path only arrives after the planner split (#247), so anything you find here gets inherited by every later slice.

What we want from you

Adversarial review, strongest objections first. It has already been through an internal 3-lens adversarial pass (secret hygiene, wire correctness, vacuous tests) — so don't stop at the obvious; assume the easy findings are taken and go for what a fresh set of eyes sees that a self-review structurally cannot.

Angles we'd specifically value:

1. **The security posture as an outsider sees it.** The token-hygiene claim ("never in argv/env/any file the crate writes") is enforced by a census test over the production source. What does that census miss? Is there a leak channel it doesn't name — stderr, panic messages, core dumps, the MCP client's own logging of tool errors?
2. **The hand-rolled choices.** Both the JSON-RPC framing and the HTTP client are hand-rolled, with rationale in doc comments. Argue against them: what real failure mode does each hand-rolled piece have that the library version wouldn't? Be concrete, not general.

3. **The MCP protocol surface.** Does the `initialize`/`tools/list`/`tools/call` handling deviate from the MCP 2024-11-05 spec in any way that will bite when a real client (Claude Desktop, Claude Code) connects?
4. **The template question.** This dispatcher is the pattern #246–#249 will copy. What's structurally missing that will hurt when the tool count goes from 1 to 10 and writes appear — cancellation, progress, concurrent tool calls on one stdio pipe, request ids colliding?

Format

One finding per line where possible: `file:line — severity — the problem — the fix`. Severities: blocker / should-fix / nit. If you verify something and it holds, say that too — "checked X, holds because Y" is as valuable as a finding. Please do not pad; a short honest list beats a long padded one.